Rapport de recherche : Bibliothèque ByzFL

Mise à l'échelle, Robustesse et Distribution Ciblée

ABBES Samy & AREZKI Célia

Semaine du 01 Juin 2026

1 Introduction

Cette semaine a été marquée par une consolidation importante de notre architecture logicielle, le passage à l'échelle de nos expériences sur Google Colab et l'implémentation de distributions de données fortement non-IID. Ces avancées nous ont permis d'évaluer la robustesse des agrégateurs sur des temps d'entraînement longs (500 rounds) et de confirmer le bon fonctionnement de notre pipeline multi-seeds et expérimenter diverses attaques byzantines.

2 Enrichissement et restructuration de la base de code

De nouveaux scénarios d'attaques et d'agrégateurs ont été ajoutés au dépôt, élargissant notre spectre de test et nos capacités d'évaluation.

Suite à la dernière réunion de suivi, nous avons corrigé les anomalies liées à la génération des graphiques. Le pipeline d'enregistrement a été entièrement refactorisé, les graphiques sont désormais générés et triés automatiquement dans des répertoires dédiés, ce qui garantit un dépôt propre et facilite grandement l'analyse des résultats.

De plus, nous avons implémenté un système d'expériences moyennées sur 10 *seeds* (graines aléatoires). Le code correspondant a été isolé dans le répertoire `expérience avec moyennes`, accompagné d'un fichier `README` explicatif détaillant son utilisation pour garantir la reproductibilité de nos expériences.

3 Déploiement sur Google Colab et validation à grande échelle

Dans un premier temps, nos tests locaux, limités à 20 rounds en raison de nos contraintes matérielles, se sont révélés peu concluants, la précision (*accuracy*) plafonnant à environ 50 %.

Pour pallier ce problème et évaluer le comportement asymptotique des modèles, nous avons mis en place un protocole d'exécution sur Google Colab. Cette transition technologique nous a permis de lancer des expériences massives sur **500 rounds** par scénario. Les résultats obtenus sont extrêmement satisfaisants, les modèles atteignent désormais des niveaux de précision très élevés, compris entre 0.96 et 0.98.

4 Analyse des attaques et performances des agrégateurs

Nous avons mené des expériences approfondies concernant l'attaque *Inner Product Manipulation* (IPM) face à différents agrégateurs. Les résultats sur 500 rounds montrent une excellente convergence globale. Les agrégateurs **Average**, **CenteredClipping**, **MoNNA** et **TrMean** montrent une progression stable, partant d'environ 0.82 au round 50 pour se stabiliser solidement au-dessus de 0.97 d'accuracy. L'agrégateur **MultiKrum** se démarque en atteignant le score le

plus haut (plus de 0.98), bien qu'il présente une instabilité temporaire remarquable (une chute soudaine vers 0.96 autour du round 450 avant de remonter).

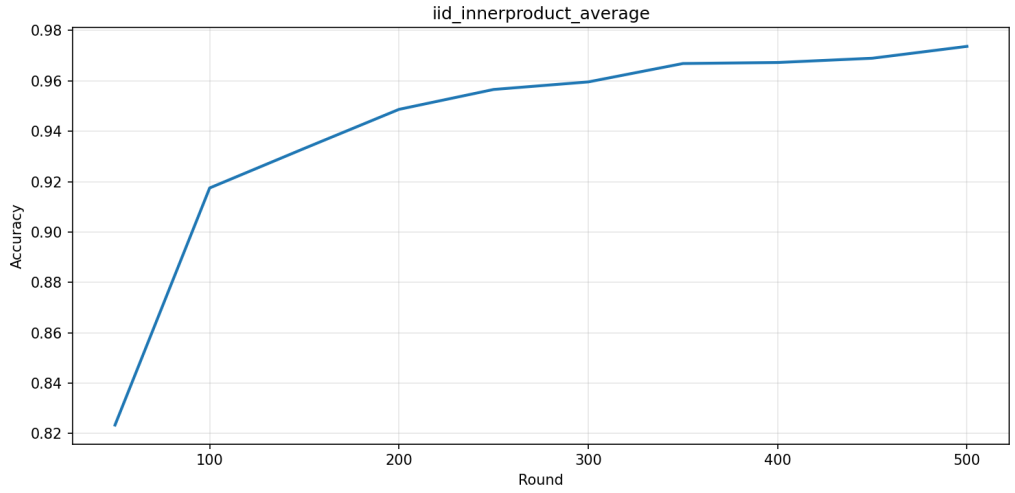


FIGURE 1 – Évolution de l'accuracy avec l'agrégateur Average face à l'attaque IPM.

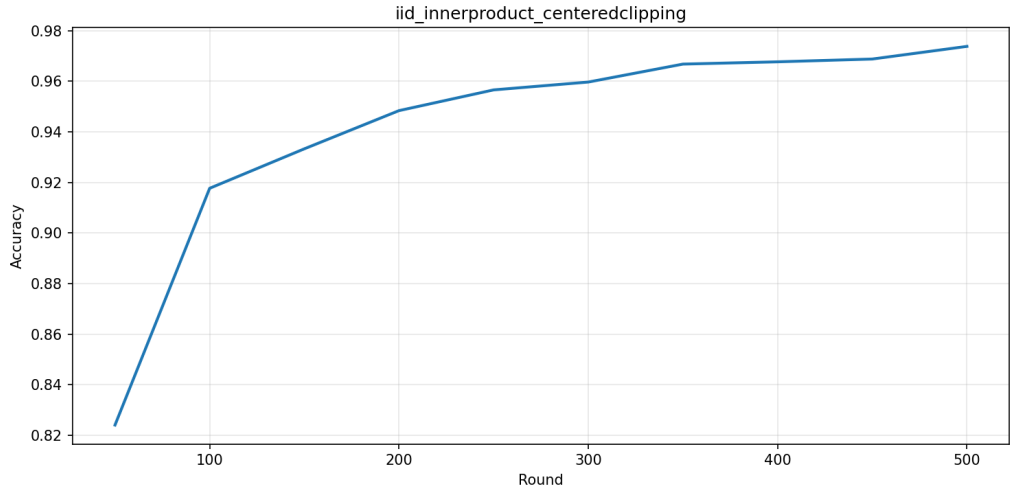


FIGURE 2 – Évolution de l'accuracy avec l'agrégateur CenteredClipping face à l'attaque IPM.

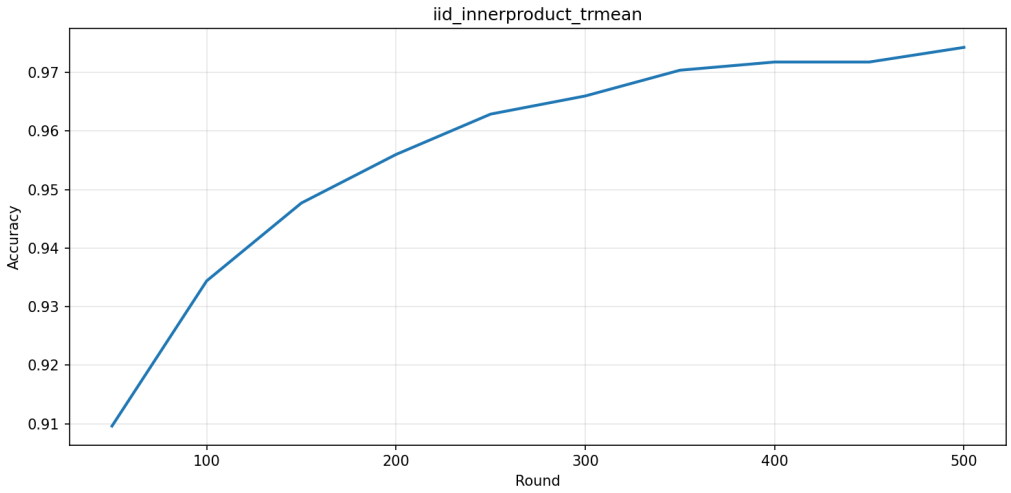


FIGURE 3 – Évolution de l'accuracy avec l'agrégateur TrMean face à l'attaque IPM.

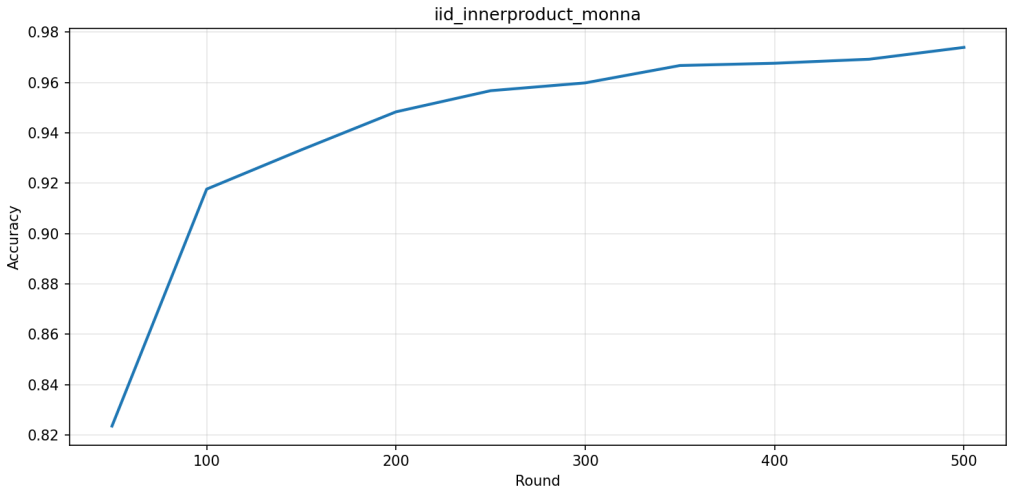


FIGURE 4 – Évolution de l'accuracy avec l'agrégateur MoNNA face à l'attaque IPM.

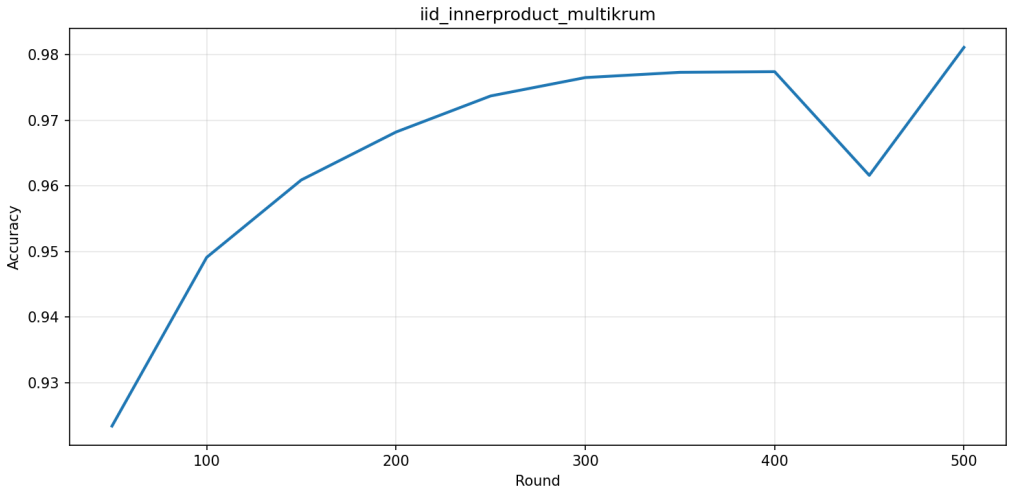


FIGURE 5 – Évolution de l'accuracy avec l'agrégateur MultiKrum face à l'attaque IPM. On note une légère instabilité autour du round 450.

Par ailleurs, nous avons également expérimenté l'attaque *Sign Flipping*. Face à cette perturbation, les agrégateurs **Average**, **Median**, **Krum** et **TrMean** se sont révélés très robustes, offrant d'excellents résultats finaux.

Afin de valider notre implémentation multi-seeds, nous avons exécuté l'attaque *Gaussian* contrée par l'agrégateur **Krum** sur 10 seeds différents. La courbe de moyenne confirme la robustesse de l'agrégateur avec une convergence vers 0.975 d'accuracy, tout en mettant en évidence l'intervalle de variation induit par les différentes initialisations.

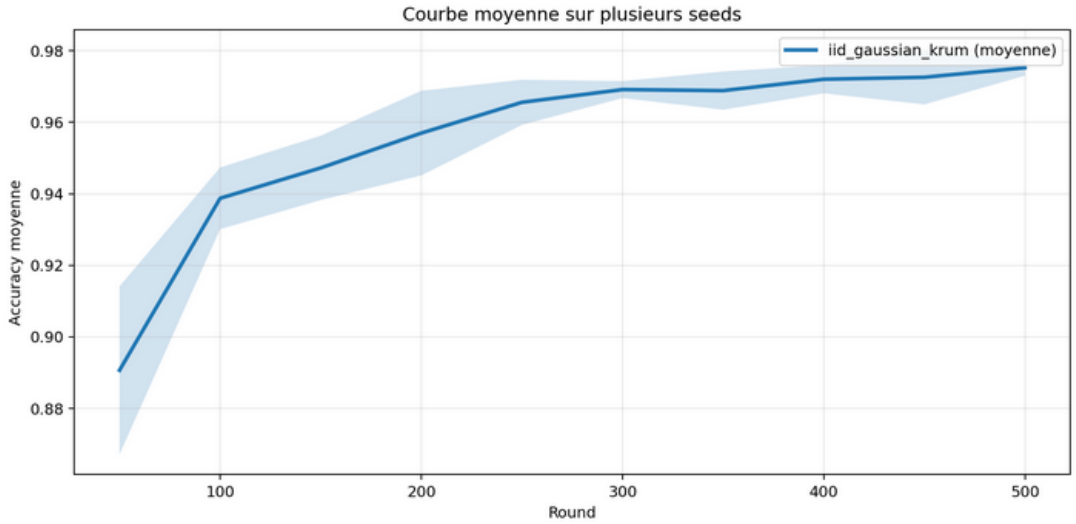


FIGURE 6 – Courbe moyenne et zone de variance sur 10 seeds pour l'attaque Gaussian avec l'agrégateur Krum.

5 Implémentation du Data Distributor (Configuration Non-IID extrême)

Conformément aux objectifs fixés pour obtenir un environnement de données hétérogènes, nous avons implémenté notre propre classe de distribution de données (*Data Distributor*). Ce

module vise à reproduire une situation de données fortement Non-IID, en forçant l'attribution d'une seule classe par client.

Bien que nous n'ayons pas encore exécuté d'expérience fédérée complète sur 500 rounds avec ce module, les premiers tests valident son parfait fonctionnement. Lors d'un test sur le dataset MNIST avec 2 clients, le distributeur a isolé les données de manière stricte, le premier client n'a reçu que des images de la classe 0, et le second uniquement des images de la classe 1 (les autres classes ayant été écartées pour ce test binaire).

La figure ci-dessous illustre visuellement le contraste entre la distribution IID native fournie par ByzFL et l'isolation stricte opérée par notre distributeur personnalisé.

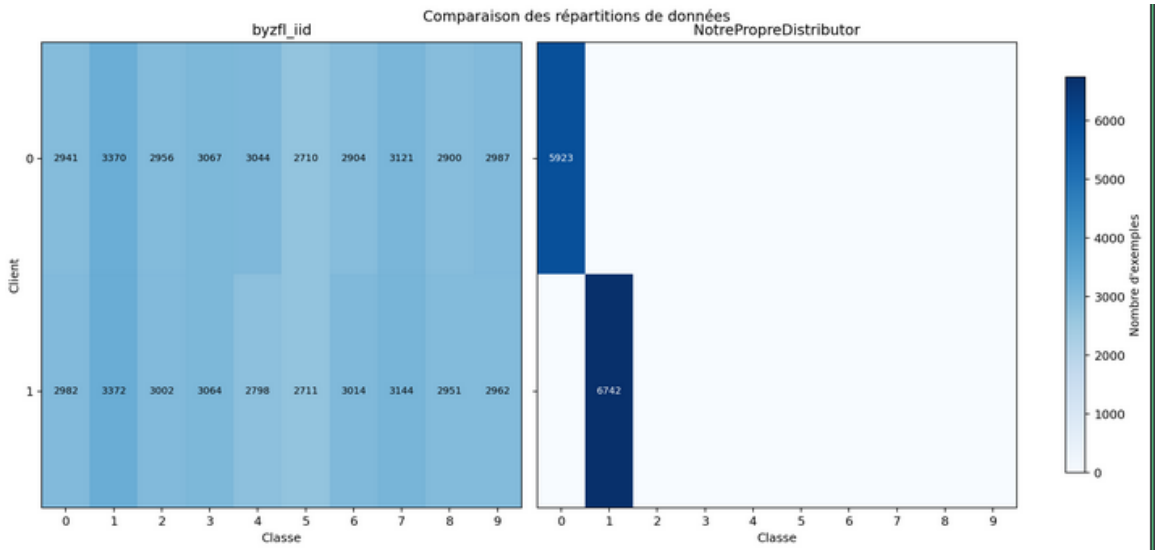


FIGURE 7 – Comparaison des répartitions de données : approche IID standard (ByzFL) à gauche vs. notre distributeur exclusif Non-IID (1 classe par client) à droite.